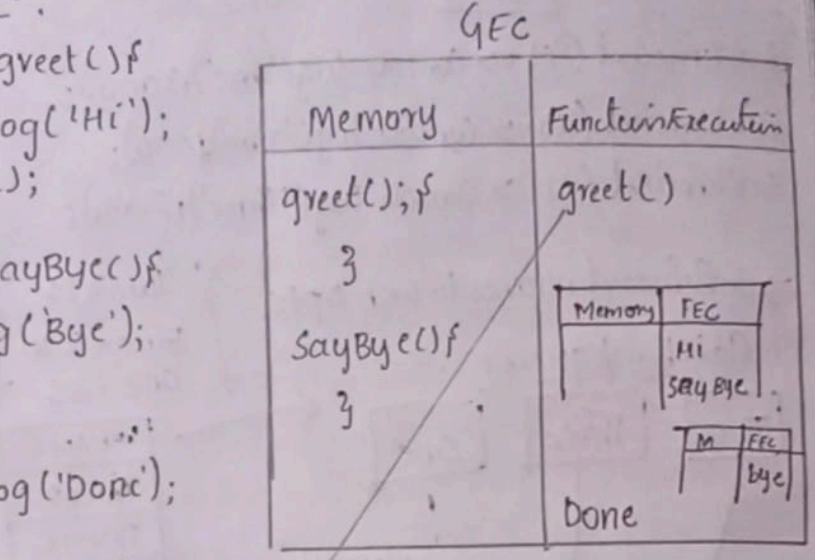
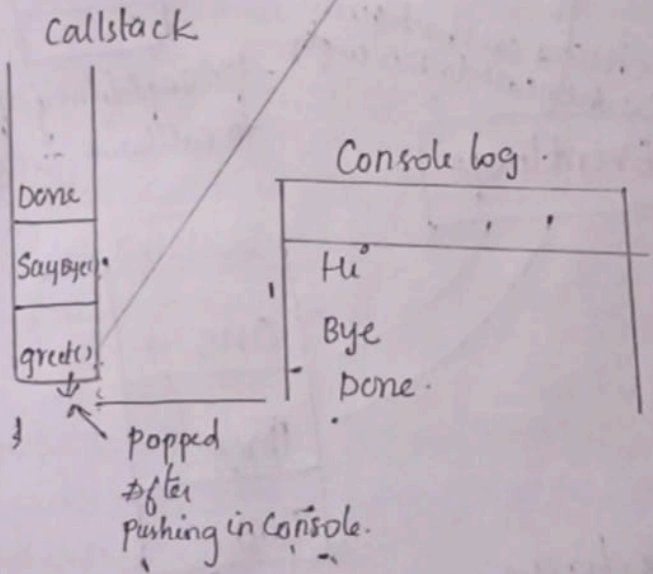
Task-1

```
function greet(){
Console.log('Hi');
 sayBye();
}
function sayBye(){
console.log('Bye');
}
greet();
Console.log('Done');
```





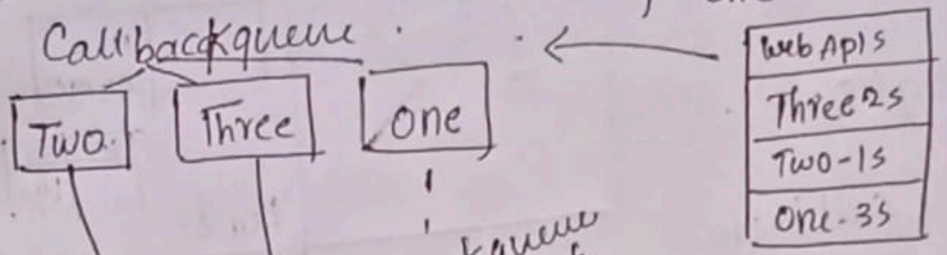
Task-2

Q.

```
setTimeOut ( () => console.log('one'),3000);
setTimeout ( ()=> console.log('two'),1000);
setTimeout ( ()=> console.log('three'),2000);
```

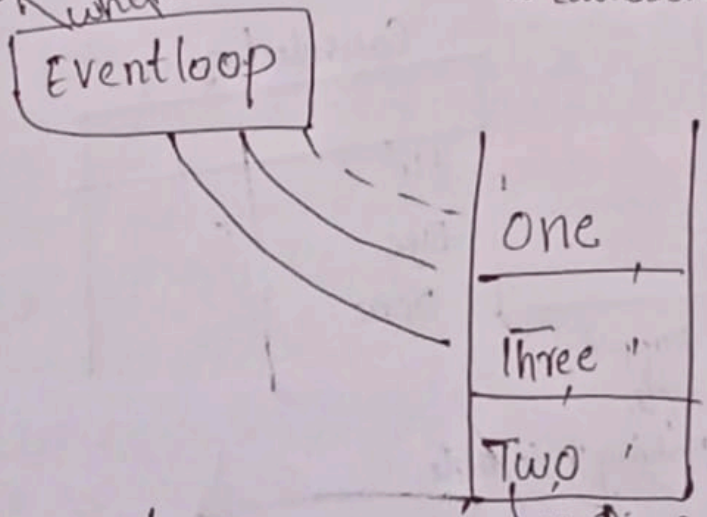
⇒ setTimeout moves to web APIs

{ two - 1s, three - 2s, one - 3s }



Eventloop checks callback queue when callstack is empty

After completeting of times moves to callback queues.



2 (1s) After
3 (2s) After
1 (3s) After

pops After Console.log from callstack.

Task 3

```
for(var i = 1; i <= 3; i++){
setTimeout(() => console.log(i), i*1000);
}
```

GEC

Memory	FEC
var i → undefined	

✓ stage 1 = loop execution

i = 1 < 3 = true setTimeout 1s)

i = 2 < 3 true setTimeout 2s)

i = 3 < 3 = true setTimeout 3s)

i = 4 < 3 = false

✓ stage 2 = Web APIs (SetTimeout move to WebAPI's After completing of timers)

Web Api

1st timer 1s	2nd timer 2s	3rd timer 3s

✓ stage 3 = callback after timer completed

Callback Queue

Timer1	Timer2	Timer3

Callbacks sit in queue, waiting for call stack is empty they execute one by one after loop is done.

✓ stage 4 = Eventloop

Eventloop checks callback queue when callstack is empty & pushes the callback to callstack.

Eventloop

Callstack

Callback3
Callback2
Callback1

Console.log

4
4
4

⇒ Single binding = 4 (After loop ends)

∴ var = 1 shared binding

point to same i = 4

∴ Each iteration gets own binding & lexical environment

Task:4 [closures - makeCounter]

```
function makeCounter() {
  let count = 0; // private variable
  return function() {
    count++;
    return count; // increment & return
  }; // inner function closes over count
} // makeCounter
```

→ Each call to makeCounter creates a new scope variable.

Each call to makeCounter() makes a new closure with its own private count.

✗ They do not share the same count.

* Create two separate counters

```
const counter1 = makeCounter(); ← new closure 1
const counter2 = makeCounter(); ← new closure 2;
```

counter1 closure

private count variable

count = 0

↓ after counter1() = count = 1

↓ after counter1() = count = 2

remember its own count, separate memory.

Outputs

counter1() = 1

counter2 closure

private count variable:

count = 0

after counter2() - count = 1

its own independent count

Output:

counter2() = 1

* Execution & Output

```
console.log(counter1()); // 1
console.log(counter1()); // 2
```

Output
1
2
1

Task 5

```
function logLater(id){
 setTimeout(function(){
 console.log("ID is"+id);
 },1000);
}
```

* call logLater three times

```
logLater(1) //creates closure id 1
logLater(2) //2
logLater(3) //3
```

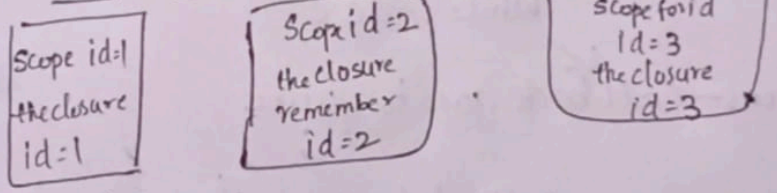
* Global execution created function store in memory then Execution process.

Execution

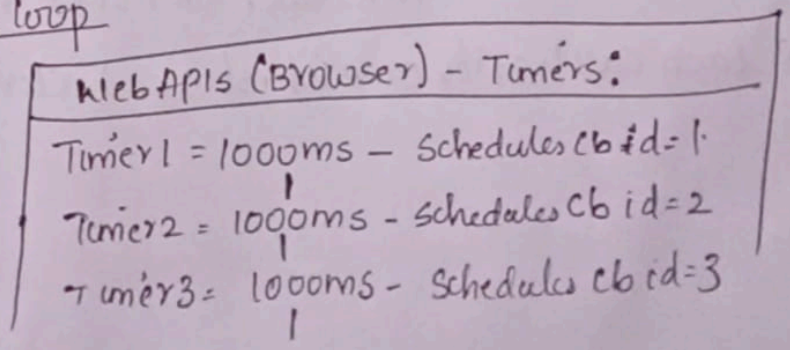
loglater(1) - runs - done ✓ - pops off
loglater(2) - runs - done ✓ - pops off
loglater(3) - runs - done ✓ - pops off

Explanation

closure - 3 Separate Scopes



Event loop

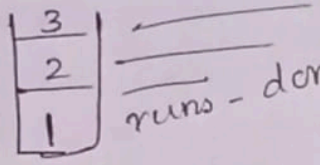


(Callback queues or Task queues is 1") -

ID-1 ID-2 ID-3 runs only when callstack is empty.

(First in, First out)

callstack



runs - done - popsoff

-After 1000ms →

output
ID is 1
ID is 2
ID 3

→ call back stack is empty now.

* Each time we call loglater(id) a new execution. Context is created & the inner function "closesOver" the id variable from the call. Even after loglater finishes and is removed from the call stack the callback function still remembers with its own id.

→ SetTimeout is (asychronous) - Send WebApis

While code ←

Continues - callback goes to queue

runs later when stack is empty

* Each call has its own id; stores in closure